Synchronization exercises

State assumptions and intermediate reasoning before consulting the separate solutions.

OS05-E01 Critical-section properties

A lock guarantees mutual exclusion but lets completed participants overtake one contender indefinitely.

1. Which property fails?
2. Does this necessarily cause deadlock?
3. Give a finite-waiting rule and bound.

OS05-E02 Bounded buffer

Capacity3, empty=0, full=3, mutex available.

1. Which semaphore must the producer wait first?
2. Give the consumer's signal order after removal.
3. Why must the producer not hold mutex while waiting empty?

OS05-E03 Condition variable

Waiter executes if(!ready) wait(cv); use(buffer). Notifier sets ready=true then signals.

1. Why can if be insufficient?
2. Give the correct waiting form.
3. How can notification before waiting avoid lost wakeup?

OS05-E04 Peterson correctness and ordering

Two threads use atomic flag[2], initially false, and turn∈{0,1}. Entry writes flag[i]=true, turn=j, then waits while(flag[j] && turn==j). Exit clears flag[i].

1. Prove exclusion by comparing the final turn writes.
2. Prove Pi can be overtaken by Pj at most once.
3. Explain all three properties under sequential consistency.
4. Which proof step fails with non-atomic variables and compiler reordering?

OS05-E05 Semaphore states

Capacity3, empty3, full0, mutex1. Produce A, B, C; consume A; produce D.

Synchronization exercises (continued)

1. Trace empty, full, mutex, and contents after each operation.
2. Why does a binary semaphore not admit a second wait?
3. If a producer blocks on empty while holding mutex, can the consumer remove an item? Give the wait edges.

OS05-E06 Mesa and lost wakeup

A monitor protects ready=false and buffer. Waiter uses while(!ready) wait(cv). Notifier holds the same monitor, sets ready=true, and signals.

1. Why must releasing the monitor and joining the queue be atomic?
2. Why is notification before predicate inspection safe?
3. Under Mesa, another thread resets ready=false before the awakened waiter reacquires. May the waiter use buffer?
4. Compare Hoare/Mesa handoff costs and correctness conditions.

OS05-E07 The notified consumer does not receive the only item

Use Mesa semantics, mutex m, condition variable cv, and initially empty q. C1/C2 each run `lock(m); while(q.empty()) wait(cv,m); pop(q); unlock(m)`. P runs `lock(m); push(q,A); signal(cv); unlock(m)` once. No spurious wakeup occurs; m is not FIFO.

Events are T0: C1 waits; T1: P holds m and inserts A; T2: P still owns m and notifies C1; T3: P unlocks, then C2 completes get and unlocks; T4: C1 reacquires m; T5: C1's next action.

1. List m owner, q, CV waiters, and notified-but-not-reacquired set E after each of six events T0–T5. The final event is C1's next action.
2. If while becomes if, identify the first invalid operation. Why is this neither a spurious wakeup nor a lost wakeup?
3. Split unlock from later wait registration. Give three steps that leave C1 asleep despite nonempty q. Is that legal pthread_cond_wait usage?
4. If both consumers are already waiting and P broadcasts after inserting one A, may both remove an item? Does while guarantee fairness or eventual completion?

References and attribution

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study